

SQL Assignment — Group 5: GROUP BY HAVING and Subqueries

Group 5 — GROUP BY, HAVING, and Subqueries

1. GROUP BY — order count per user

Show the total number of orders placed by each user in an 'orders' table, displaying `user_id` and `order_count`.

```
SELECT user_id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

Explanation: `GROUP BY user_id` groups all order rows by the user who placed them, and `COUNT(*)` counts how many rows (orders) fall into each group, giving one row per user with their total order count.

2. GROUP BY + HAVING — restaurants with avg rating > 4.0

Display the names of restaurants that have received an average rating above 4.0, using `GROUP BY` and `HAVING`.

```
-- Assumes a 'restaurants' table where each row is one rating/review for a restaurant,
-- with columns like restaurant_name and rating.
SELECT restaurant_name, AVG(rating) AS avg_rating
FROM restaurants
GROUP BY restaurant_name
HAVING AVG(rating) > 4.0;
```

Explanation: `GROUP BY restaurant_name` groups all rating rows per restaurant, and `AVG(rating)` computes the average within each group. `HAVING` is used instead of `WHERE` here because the filter (`AVG(rating) > 4.0`) applies to an aggregated value — `WHERE` filters individual rows before grouping, while `HAVING` filters the grouped results after aggregation.

3. Subquery — users with a single payment above ₹2000

Given a 'payments' table (`user_id`, `amount`, `payment_date`), write a subquery to find `user_ids` who have made a single payment above ₹2000.

```
SELECT DISTINCT user_id
FROM payments
WHERE user_id IN (
    SELECT user_id
    FROM payments
    WHERE amount > 2000
);
```

Explanation: The inner subquery finds every `user_id` associated with any payment row where `amount > 2000`. The outer query then selects those same `user_ids` (with `DISTINCT` to avoid duplicates if a user made more than one large payment). In this particular case, since the subquery already returns exactly the right `user_ids`, a simpler equivalent would be `SELECT DISTINCT user_id FROM payments WHERE amount > 2000;` — the subquery form is shown here to demonstrate the technique as requested.

4. Nested subquery — movies rated above the overall average

Display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

```
-- Assumes a 'movies' table with columns: movie_name, rating
SELECT movie_name, rating
FROM movies
WHERE rating > (
    SELECT AVG(rating)
    FROM movies
);
```

Explanation: The inner subquery `SELECT AVG(rating) FROM movies` calculates a single value: the average rating across every movie in the table. The outer query then compares each individual movie's `rating` against that one overall average, returning only the movies whose rating exceeds it.